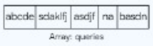
Sparse Arrays | HackerRank　　Dynamic array solution C

hackerrank.com/challenges/sparse-arrays/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar　Set as default

HackerRank | Prepare > Data Structures > Arrays > Sparse Arrays　　Exit Full Screen View

Subm　Leaderboard　Discussions　Editorial



Sample Output 3

Change Theme　Language　C

```c
int main() {

    char queries[q][101];
    for(int i = 0; i < q; i++) {
        scanf("%s", queries[i]);
    }

    // For each query, count occurrences
    for(int i = 0; i < q; i++) {
        int count = 0;
        for(int j = 0; j < n; j++) {
            if(strcmp(queries[i], strings[j]) == 0) {
                count++;
            }
        }
        printf("%d\n", count);
    }

    return 0;
}
```

Line: 34 Col: 1

Upload Code as File　Test against custom input　Run Code　Submit Code

Sparse Arrays | HackerRank
Dynamic array solution C

hackerrank.com/challenges/sparse-arrays/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar
Set as default

HackerRank | Prepare > Data Structures > Arrays > Sparse Arrays
Exit Full Screen View

Subm
Leaderboard
Discussions
Editorial

abcde	sdaklfj	asdjf	na	basdn

Array: queries

Sample Output 3

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Sample Test case 2

Input (stdin)
Download

```
4
aba
baba
aba
xzxb
3
aba
xzxb
ab
```

Your Output (stdout)

```
2
```

Array Manipulation | HackerRank × Dynamic array solution C ×

hackerrank.com/challenges/crush/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar Set as default

HackerRank | Prepare > Data Structures > Arrays > Array Manipulation Exit Full Screen View

- $1 \leq a \leq b \leq n$
- $0 \leq k \leq 10^9$

Sample Input

```
STDIN      Function
-----      --------
5 3       arr[] size n = 5, queries[] size q = 3
1 2 100    queries = [[1, 2, 100], [2, 5, 100], [3, 4, 100]]
2 5 100
3 4 100
```

Sample Output

```
200
```

Explanation

After the first update the list is `100 100 0 0 0`.

After the second update list is `100 200 100 100 100`.

After the third update list is `100 200 200 200 100`.

The maximum value is 200.

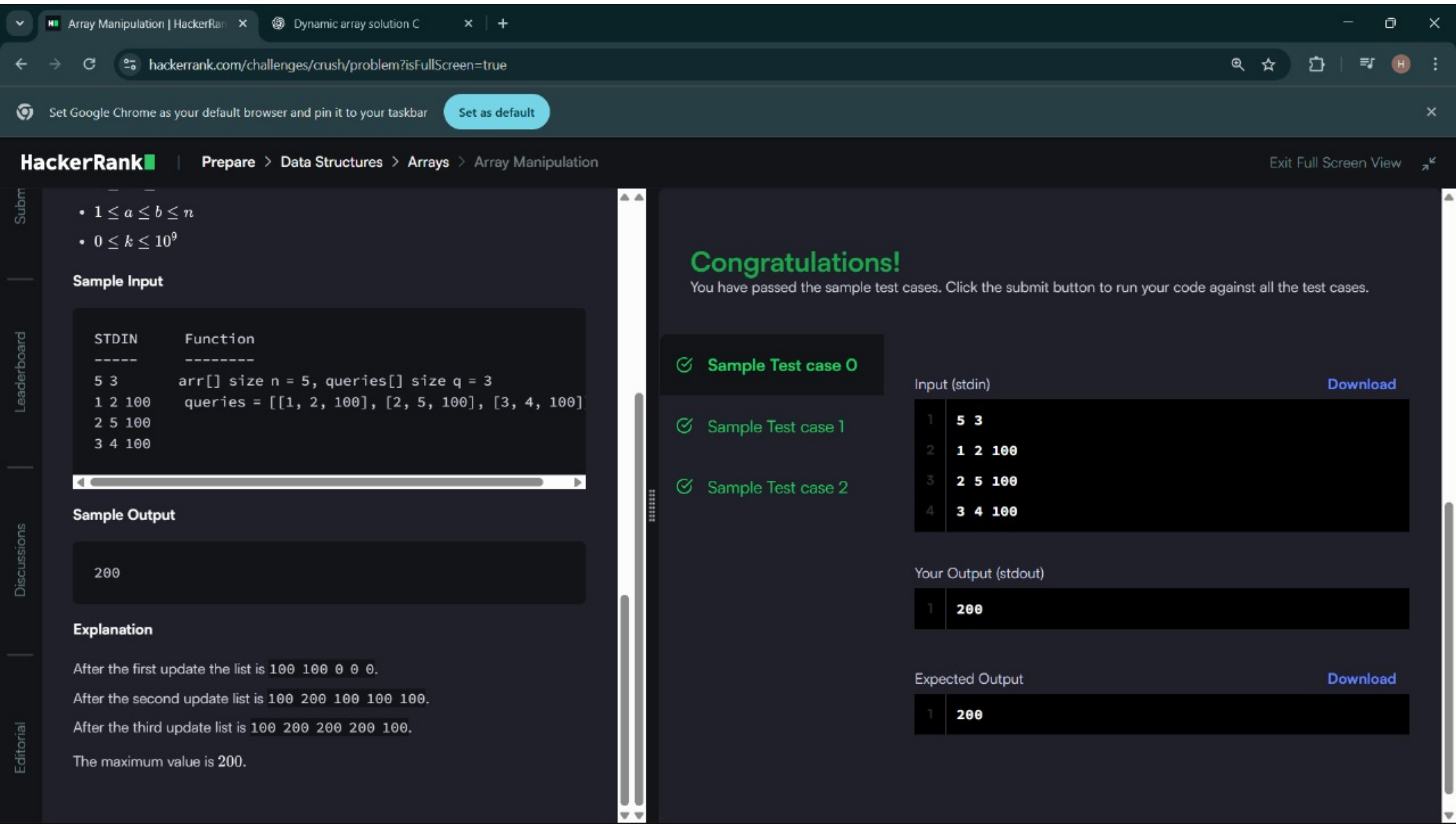
Change Theme Language C

```c
int main() {
        scanf("%lld %lld %lld", &a, &b, &k);

        arr[a] += k;      // Add k at start
        arr[b + 1] -= k;  // Subtract k after end
    }

    long long max = 0, current = 0;

    for (long long i = 1; i <= n; i++) {
        current += arr[i];    // Prefix sum reconstructs array
        if (current > max)
            max = current;
    }

    printf("%lld\n", max);

    free(arr);
    return 0;
}
```

Line: 31 Col: 1

Battery status: 78% remaining

27°C Mostly cloudy Search ENG IN 18:27 21-11-2025



- $1 \leq a \leq b \leq n$
- $0 \leq k \leq 10^9$

Sample Input

```
STDIN       Function
-----       --------
5 3        arr[] size n = 5, queries[] size q = 3
1 2 100     queries = [[1, 2, 100], [2, 5, 100], [3, 4, 100]]
2 5 100
3 4 100
```

Sample Output

```
200
```

Explanation

After the first update the list is `100 100 0 0 0`.

After the second update list is `100 200 100 100 100`.

After the third update list is `100 200 200 200 100`.

The maximum value is 200.

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Sample Test case 1

Sample Test case 2

Input (stdin) Download

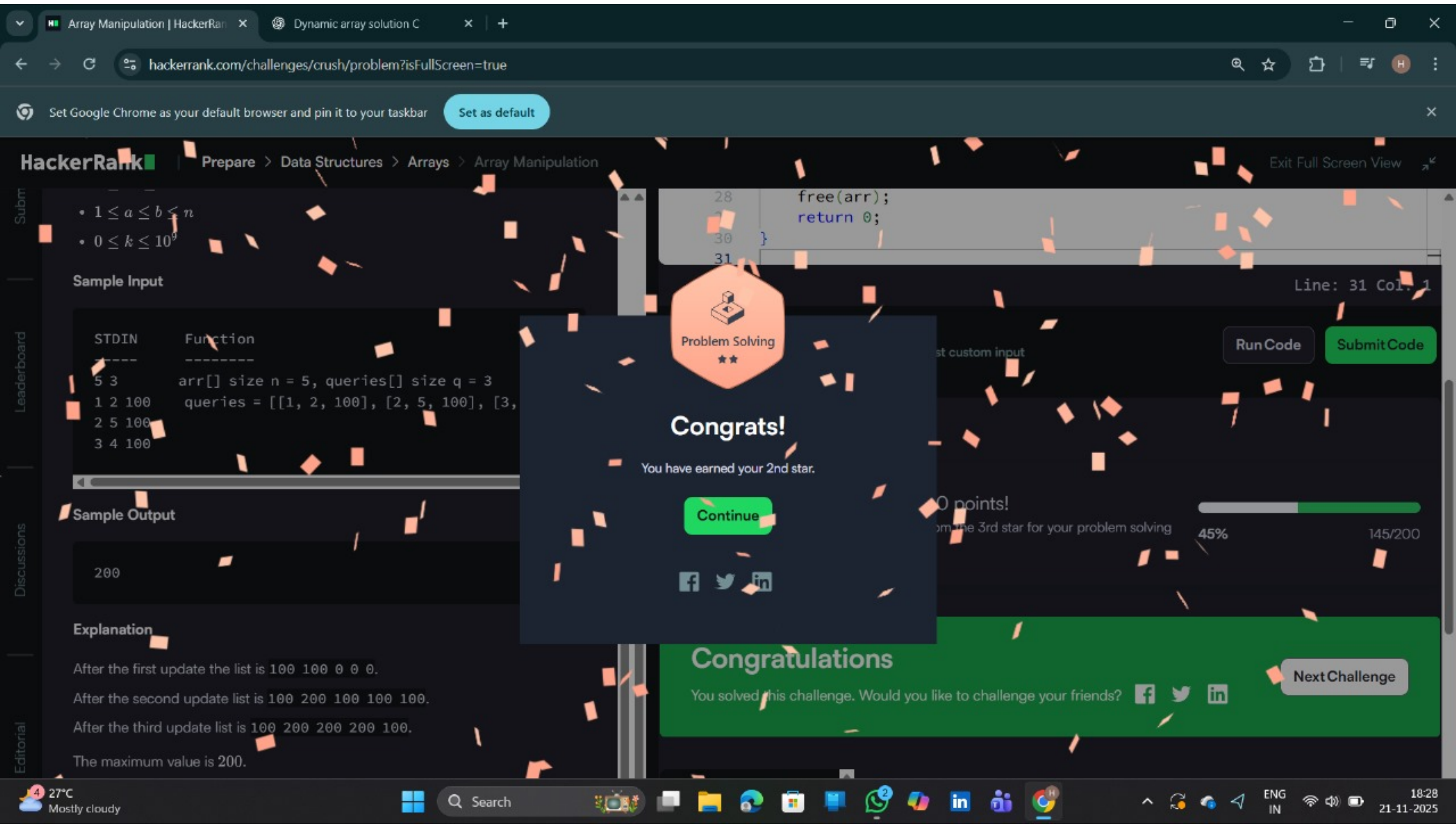
```
5 3
1 2 100
2 5 100
3 4 100
```

Your Output (stdout)

```
200
```

Expected Output Download

```
200
```


Array Manipulation | HackerRan
Dynamic array solution C
hackerrank.com/challenges/crush/problem?isFullScreen=true
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
HackerRank
Prepare > Data Structures > Arrays > Array Manipulation
Exit Full Screen View
• 1 ≤ a ≤ b ≤ n
• 0 ≤ k ≤ 10^9
Sample Input
STDIN Function
5 3 arr[] size n = 5, queries[] size q = 3
1 2 100 queries = [[1, 2, 100], [2, 5, 100], [3,
2 5 100
3 4 100
Sample Output
200
Explanation
After the first update the list is 100 100 0 0 0.
After the second update list is 100 200 100 100 100.
After the third update list is 100 200 200 200 100.
The maximum value is 200.
free(arr);
return 0;
}
Line: 31 Col: 1
Run Code
Submit Code
Problem Solving
Congrats!
You have earned your 2nd star.
Continue
points!
the 3rd star for your problem solving
45%
145/200
Congratulations
You solved this challenge. Would you like to challenge your friends?
Next Challenge
27°C
Mostly cloudy
Search
ENG
IN
18:28
21-11-2025

Jim and the Skyscrapers | Hack × Dynamic array solution C ×

hackerrank.com/challenges/jim-and-the-skyscrapers/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar Set as default

HackerRank | Prepare > Data Structures > Advanced > Jim and the Skyscrapers Exit Full Screen View

Subm | Leaderboard | Discussions | Editorial

```
6
3 2 1 2 3 3
```

Sample Output #00

```
8
```

Sample Input #01

```
3
1 1000 1
```

Sample Output #01

```
0
```

Explanation

First testcase: (1, 5), (1, 6) (5, 6) and (2, 4) and the routes in the opposite directions are the only valid routes.

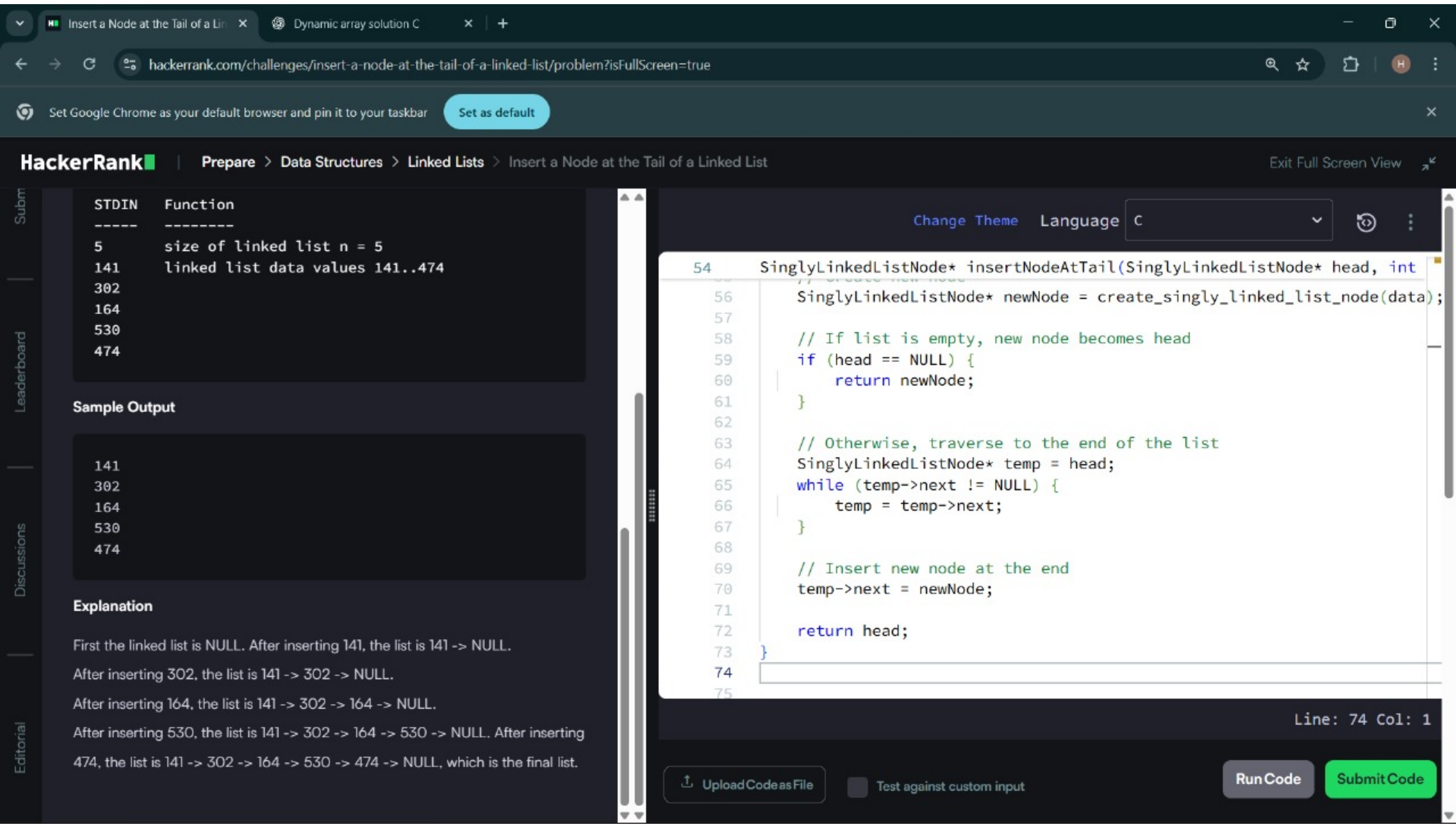
Second testcase: (1, 3) and (3, 1) could have been valid, if there wasn't a big skyscraper with height 1000 between them.

Change Theme Language C

```c
int main() {
        while (top >= 0 && sh[top] < h[i]) {
            top--;
        }

        if (top >= 0 && sh[top] == h[i]) {
            // Same height -> valid pairs formed
            ans += freq[top];
            freq[top]++;   // Increase frequency
        } else {
            // New height entry
            top++;
            sh[top] = h[i];
            freq[top] = 1;
        }
    }

    printf("%lld\n", ans);
    return 0;
}
```

Line: 41 Col: 1

Upload Code as File Test against custom input Run Code Submit Code


Insert a Node at the Tail of a Lin
Dynamic array solution C
hackerrank.com/challenges/insert-a-node-at-the-tail-of-a-linked-list/problem?isFullScreen=true
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
HackerRank
Prepare > Data Structures > Linked Lists > Insert a Node at the Tail of a Linked List
Exit Full Screen View
STDIN Function
----- --------
5 size of linked list n = 5
141 linked list data values 141..474
302
164
530
474
Sample Output
141
302
164
530
474
Explanation
First the linked list is NULL. After inserting 141, the list is 141 -> NULL.
After inserting 302, the list is 141 -> 302 -> NULL.
After inserting 164, the list is 141 -> 302 -> 164 -> NULL.
After inserting 530, the list is 141 -> 302 -> 164 -> 530 -> NULL. After inserting 474, the list is 141 -> 302 -> 164 -> 530 -> 474 -> NULL, which is the final list.
Change Theme
Language
C
SinglyLinkedListNode* insertNodeAtTail(SinglyLinkedListNode* head, int
SinglyLinkedListNode* newNode = create_singly_linked_list_node(data);
// If list is empty, new node becomes head
if (head == NULL) {
return newNode;
}
// Otherwise, traverse to the end of the list
SinglyLinkedListNode* temp = head;
while (temp->next != NULL) {
temp = temp->next;
}
// Insert new node at the end
temp->next = newNode;
return head;
}
Line: 74 Col: 1
Upload Code as File
Test against custom input
Run Code
Submit Code

Insert a Node at the Tail of a Lin ×   Dynamic array solution C ×

hackerrank.com/challenges/insert-a-node-at-the-tail-of-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar   Set as default

HackerRank | Prepare > Data Structures > Linked Lists > Insert a Node at the Tail of a Linked List   Exit Full Screen View

```
STDIN   Function
-----   --------
5       size of linked list n = 5
141     linked list data values 141..474
302
164
530
474
```

Sample Output

```
141
302
164
530
474
```

Explanation

First the linked list is NULL. After inserting 141, the list is 141 -> NULL.

After inserting 302, the list is 141 -> 302 -> NULL.

After inserting 164, the list is 141 -> 302 -> 164 -> NULL.

After inserting 530, the list is 141 -> 302 -> 164 -> 530 -> NULL. After inserting 474, the list is 141 -> 302 -> 164 -> 530 -> 474 -> NULL, which is the final list.

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

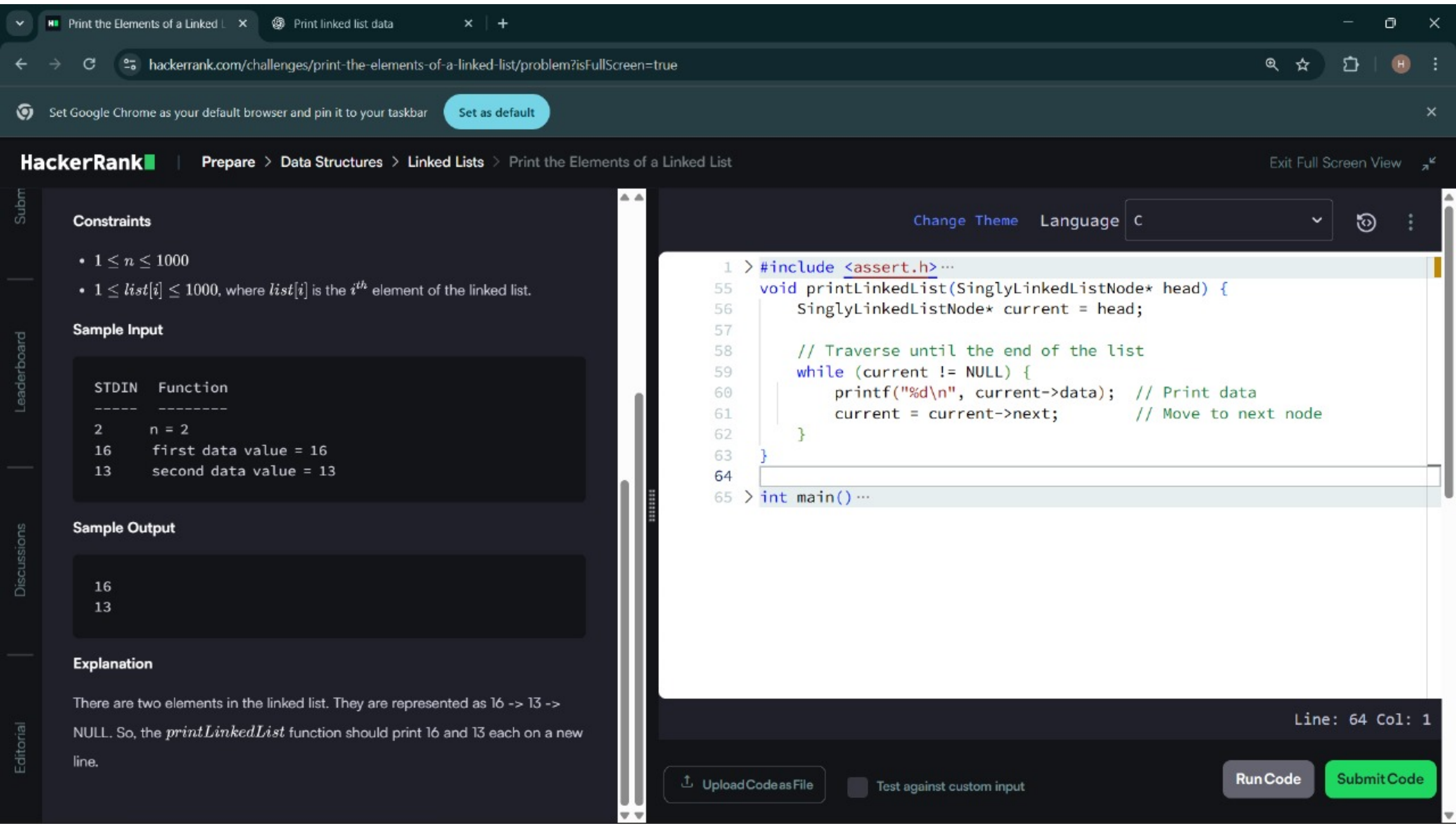
Sample Test case 1

Input (stdin)   Download

```
5
141
302
164
530
474
```

Your Output (stdout)

```
141
302
164
530
```


Print the Elements of a Linked L
Print linked list data
hackerrank.com/challenges/print-the-elements-of-a-linked-list/problem?isFullScreen=true
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
HackerRank
Prepare > Data Structures > Linked Lists > Print the Elements of a Linked List
Exit Full Screen View
Constraints
• $1 \le n \le 1000$
• $1 \le list[i] \le 1000$, where $list[i]$ is the i^{th} element of the linked list.
Sample Input
STDIN Function
----- --------
2 n = 2
16 first data value = 16
13 second data value = 13
Sample Output
16
13
Explanation
There are two elements in the linked list. They are represented as 16 -> 13 -> NULL. So, the $printLinkedList$ function should print 16 and 13 each on a new line.
Change Theme
Language
C
#include <assert.h>···
void printLinkedList(SinglyLinkedListNode* head) {
SinglyLinkedListNode* current = head;
// Traverse until the end of the list
while (current != NULL) {
printf("%d\n", current->data); // Print data
current = current->next; // Move to next node
}
}
int main()···
Line: 64 Col: 1
Upload Code as File
Test against custom input
Run Code
Submit Code
Submissions
Leaderboard
Discussions
Editorial

Delete a Node | HackerRank

hackerrank.com/challenges/delete-a-node-from-a-linked-list/problem?isFullScreen=true

Problem | Submissions | Leaderboard

Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.

Example

$llist = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

$position = 2$

After removing the node at position 2, $llist' = 0 \rightarrow 1 \rightarrow 3$.

Function Description

Complete the deleteNode function in the editor below.

deleteNode has the following parameters:

- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove

Returns

- SinglyLinkedListNode pointer: a reference to the head of the modified list

Input Format

The first line of input contains an integer n, the number of elements in the linked list.

Each of the next n lines contains an integer, the node data values in order.

The last line contains an integer, $position$, the position of the node to delete.

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

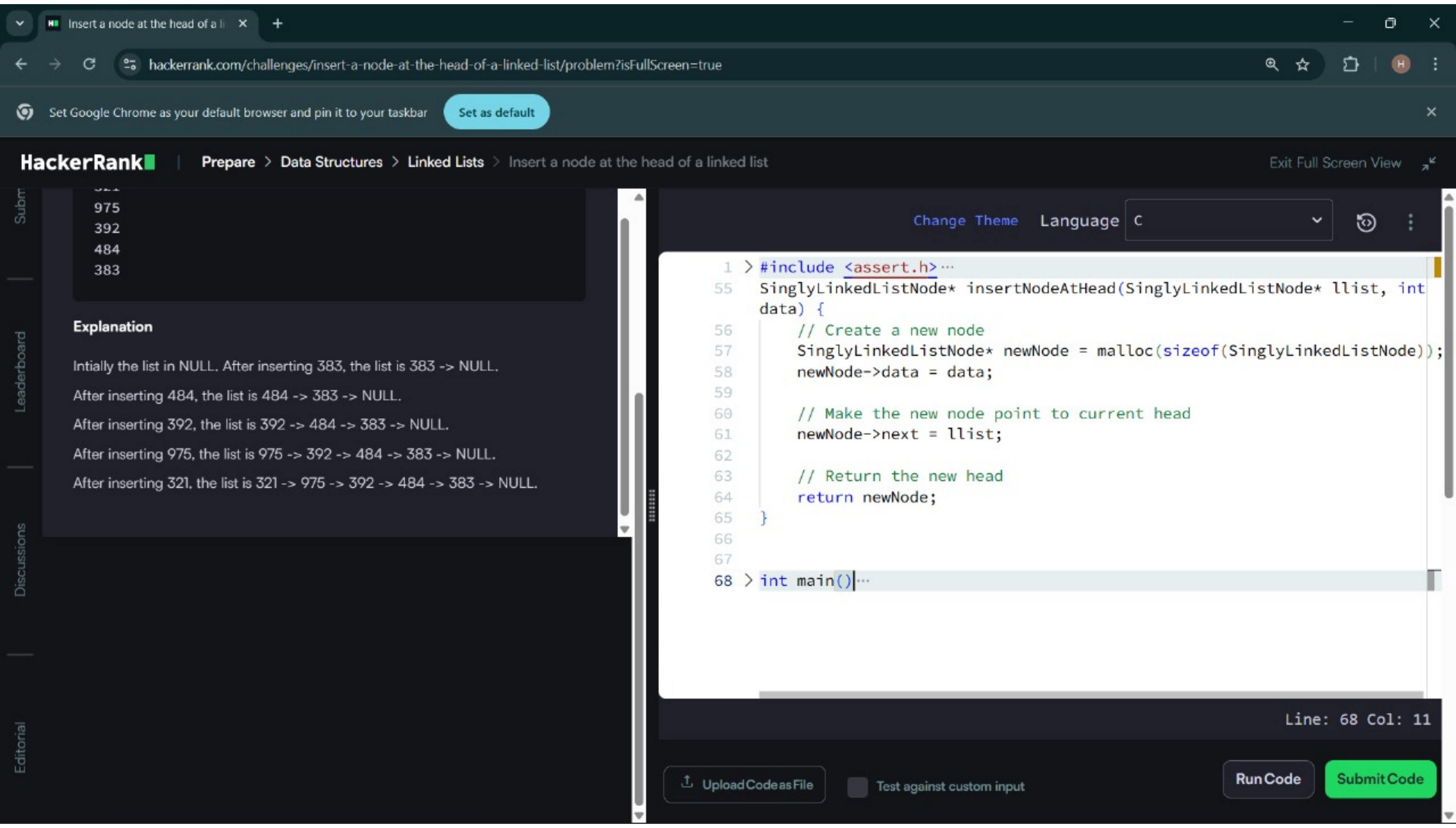
Sample Test case 0

Sample Test case 1

Input (stdin)   Download

```
8
20
6
2
19
7
4
15
9
3
```

Your Output (stdout)


Insert a node at the head of a li
hackerrank.com/challenges/insert-a-node-at-the-head-of-a-linked-list/problem?isFullScreen=true
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
HackerRank
Prepare > Data Structures > Linked Lists > Insert a node at the head of a linked list
Exit Full Screen View
Subm
Leaderboard
Discussions
Editorial
975
392
484
383
Explanation
Intially the list in NULL. After inserting 383, the list is 383 -> NULL.
After inserting 484, the list is 484 -> 383 -> NULL.
After inserting 392, the list is 392 -> 484 -> 383 -> NULL.
After inserting 975, the list is 975 -> 392 -> 484 -> 383 -> NULL.
After inserting 321, the list is 321 -> 975 -> 392 -> 484 -> 383 -> NULL.
Change Theme
Language
C
#include <assert.h>···
SinglyLinkedListNode* insertNodeAtHead(SinglyLinkedListNode* llist, int data) {
// Create a new node
SinglyLinkedListNode* newNode = malloc(sizeof(SinglyLinkedListNode));
newNode->data = data;
// Make the new node point to current head
newNode->next = llist;
// Return the new head
return newNode;
}
int main()···
Line: 68 Col: 11
Upload Code as File
Test against custom input
Run Code
Submit Code

Insert a node at a specific posit

hackerrank.com/challenges/insert-a-node-at-a-specific-position-in-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar Set as default

HackerRank | Prepare > Data Structures > Linked Lists > Insert a node at a specific position in a linked list

Exit Full Screen View

Problem | Submissions | Leaderboard

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head node of a linked list and an integer to insert at a certain position, create a new node with the given integer as its $data$ attribute, insert this node at the desired position, and return the head node.

A position of 0 indicates the head, a position of 1 indicates one node away from the head, and so on. The head pointer given may be null, meaning that the initial list is empty.

Example

$head$ refers to the first node in the list $1 \rightarrow 2 \rightarrow 3$

$data = 4$

$position = 2$

Insert a node at position 2 with $data = 4$. The new list is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Function Description

Complete the function $insertNodeAtPosition$ with the following parameters:

- $SinglyLinkedListNode\ pointer\ llist$: a reference to the head of the list
- $data$: an integer value to insert as data in the new node
- $position$: an integer position to insert the new node, zero-based indexing

Returns

- $SinglyLinkedListNode\ pointer$: a reference to the head of the revised list

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

- Sample Test case 0
- Sample Test case 1
- Sample Test case 2

Input (stdin) Download

```
3
16
13
7
1
2
```

Your Output (stdout)

```
16 13 1 7
```

Expected Output Download

```
16 13 1 7
```

Insert a node at a specific posit ×

hackerrank.com/challenges/insert-a-node-at-a-specific-position-in-a-linked-list/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar   Set as default

HackerRank | Prepare > Data Structures > Linked Lists > Insert a node at a specific position in a linked list   Exit Full Screen View

Problem | Submissions | Leaderboard

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head node of a linked list and an integer to insert at a certain position, create a new node with the given integer as its $data$ attribute, insert this node at the desired position, and return the head node.

A position of 0 indicates the head, a position of 1 indicates one node away from the head, and so on. The head pointer given may be null, meaning that the initial list is empty.

Example

$head$ refers to the first node in the list $1 \rightarrow 2 \rightarrow 3$

$data = 4$

$position = 2$

Insert a node at position 2 with $data = 4$. The new list is $1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Function Description

Complete the function $insertNodeAtPosition$ with the following parameters:

- $SinglyLinkedListNode\ pointer\ llist$: a reference to the head of the list
- $data$: an integer value to insert as data in the new node
- $position$: an integer position to insert the new node, zero-based indexing

Returns

- $SinglyLinkedListNode\ pointer$: a reference to the head of the revised list

Change Theme   Language   C

```c
SinglyLinkedListNode* insertNodeAtPosition(

    // Case 2: Insert at given position (middle or end)
    SinglyLinkedListNode* current = head;
    int index = 0;

    // Traverse to the node before the insertion point
    while (index < position - 1 && current != NULL) {
        current = current->next;
        index++;
    }

    // Now 'current' points to (position-1)th node
    newNode->next = current->next;
    current->next = newNode;

    // Return unchanged head
    return head;
}
```

Line: 100 Col: 1

Upload Code as File   Test against custom input   Run Code   Submit Code

Delete a Node | HackerRank

hackerrank.com/challenges/delete-a-node-from-a-linked-list/problem?isFullScreen=true

Problem | Submissions | Leaderboard

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.

Example

$llist = 0 \to 1 \to 2 \to 3$

$position = 2$

After removing the node at position 2, $llist' = 0 \to 1 \to 3$.

Function Description

Complete the deleteNode function in the editor below.

deleteNode has the following parameters:

- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove

Returns

- SinglyLinkedListNode pointer: a reference to the head of the modified list

Input Format

The first line of input contains an integer n, the number of elements in the linked list.

Each of the next n lines contains an integer, the node data values in order.

Change Theme   Language C

```c
#include <assert.h>...
SinglyLinkedListNode* deleteNode(SinglyLinkedListNode* llist, int
position) {

    // Case 1: Delete head node (position = 0)
    if (position == 0) {
        SinglyLinkedListNode* temp = llist;   // current head
        llist = llist->next;                  // move head forward
        free(temp);                           // free old head
        return llist;                         // return new head
    }

    // Case 2: Delete node in middle or end
    SinglyLinkedListNode* current = llist;
    int index = 0;

    // Traverse to one node before the node to delete
    while (index < position - 1 && current != NULL) {
        current = current->next;
        index++;
    }
```

Line: 67 Col: 1

Upload Code as File   Test against custom input   Run Code   Submit Code

Print in Reverse | HackerRank

hackerrank.com/challenges/print-the-elements-of-a-linked-list-in-reverse/problem?isFullScreen=true

Set Google Chrome as your default browser and pin it to your taskbar   Set as default

HackerRank | Prepare > Data Structures > Linked Lists > Print in Reverse   Exit Full Screen View

Problem | Submissions | Leaderboard

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head of a singly-linked list, print each $data$ value from the reversed list. If the given list is empty, do not print anything.

Example

$head*$ refers to the linked list with $data$ values $1 \rightarrow 2 \rightarrow 3 \rightarrow NULL$

Print the following:

3

2

1

Function Description

Complete the reversePrint function in the editor below.

reversePrint has the following parameters:

- SinglyLinkedListNode pointer head: a reference to the head of the list

Prints

The $data$ values of each node in the reversed list.

Input Format

The first line of input contains t, the number of test cases.

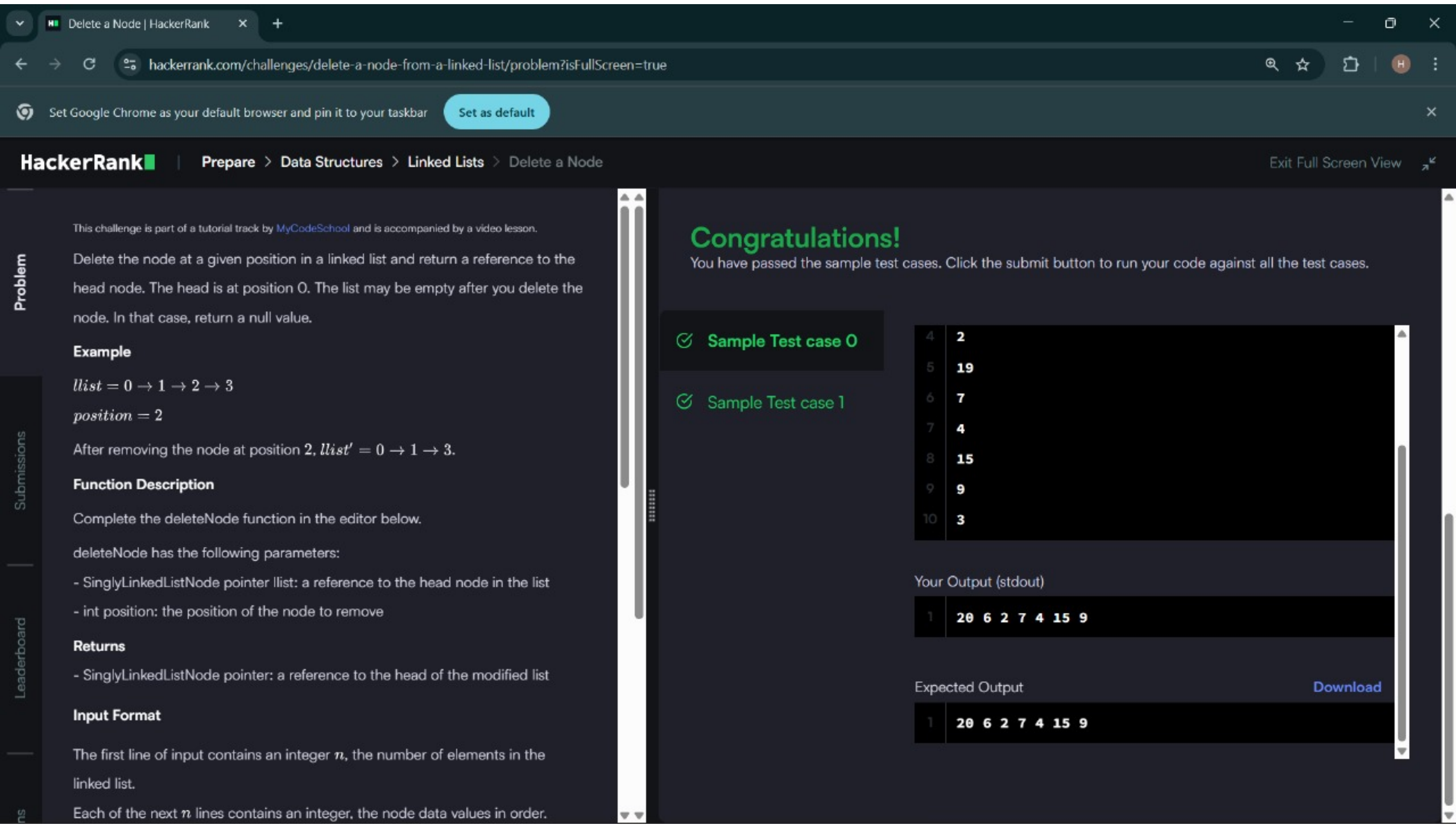
The input of each test case is as follows:

Change Theme   Language C

```c
#include <assert.h>···

void reversePrint(SinglyLinkedListNode* head) {
    if (head == NULL) return;       // base case: empty list
    reversePrint(head->next);       // recursively print the rest of the list
    printf("%d\n", head->data);     // print current node after recursion
}


int main()···
```

Line: 67 Col: 1

Upload Code as File   Test against custom input   Run Code   Submit Code



Delete a Node | HackerRank
hackerrank.com/challenges/delete-a-node-from-a-linked-list/problem?isFullScreen=true
Set Google Chrome as your default browser and pin it to your taskbar
Set as default
HackerRank
Prepare > Data Structures > Linked Lists > Delete a Node
Exit Full Screen View
Problem
Submissions
Leaderboard
This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.
Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.
Example
$llist = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$
$position = 2$
After removing the node at position 2, $llist' = 0 \rightarrow 1 \rightarrow 3$.
Function Description
Complete the deleteNode function in the editor below.
deleteNode has the following parameters:
- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove
Returns
- SinglyLinkedListNode pointer: a reference to the head of the modified list
Input Format
The first line of input contains an integer n, the number of elements in the linked list.
Each of the next n lines contains an integer, the node data values in order.
Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.
Sample Test case 0
Sample Test case 1
4 2
5 19
6 7
7 4
8 15
9 9
10 3
Your Output (stdout)
1 20 6 2 7 4 15 9
Expected Output
Download
1 20 6 2 7 4 15 9